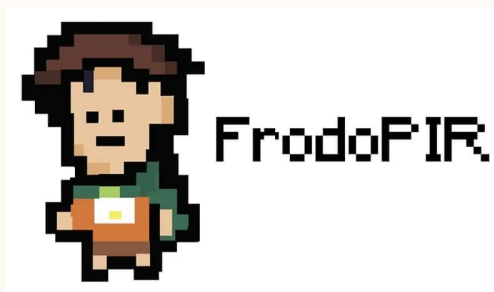


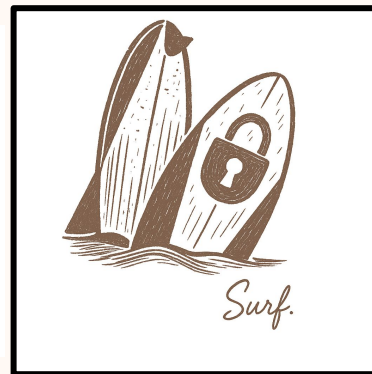
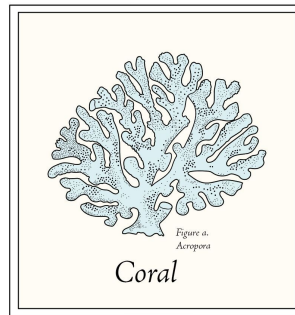
PRACTICAL CRYPTOGRAPHY

crypt@b-it 2026
Summer school on Cryptography
Sofía Celi



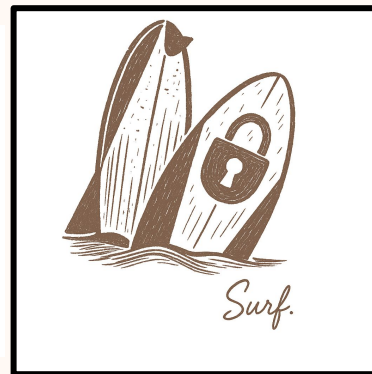
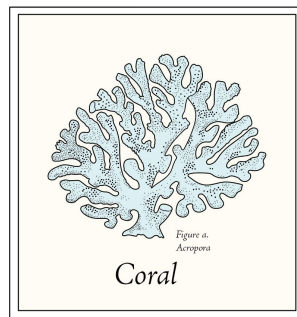
LECTURE 4:

ADVANCES AND LIMITS IN ZKP-BASED SYSTEMS FOR ATTESTATION



LECTURE 4:

ADVANCES AND LIMITS IN ZKP-BASED SYSTEMS FOR ATTESTATION: OR HOW TO MPC TLS



Let's think about this

Today, we are going to see a not-so-practical scheme:

- We are going to look at it as a counterexample to TLS 1.3
- Remember: TLS 1.3 is quite complex in itself, but today we will see how we can make it more complex ;)
- We are going to try to ask: why do we need this complexity

Outline

This lecture will cover multiple works (and we will see why)

- Where all started: “DiStefano: Decentralized Infrastructure for Sharing Trusted Encrypted Facts and Nothing More”: <https://eprint.iacr.org/2023/1063>
- “CDLS: Proving Knowledge of Committed Discrete Logarithms with Soundness”: <https://eprint.iacr.org/2023/1595>
- “Coral: Fast Succinct Non-Interactive Zero-Knowledge CFG Proofs”: <https://eprint.iacr.org/2025/1420.pdf>
- “The limits of zero-knowledge for age-verification”: <https://brave.com/blog/zkp-age-verification-limits/> (but also: <https://www.eff.org/deeplinks/2025/07/zero-knowledge-proofs-alone-are-not-digital-id-solution-protecting-user-privacy> <https://www.iab.org/announcements/call-for-papers-workshop-on-age-based-restrictions-on-content-access/>)
- Upcoming work on *attacks on* systems and attesting authorization over the web

Outline

The design of a complex/distributed system

- How to commit to encrypted data sent over a channel: data that provides certain assurances
- How to prove **in zkp** that said data has certain statements:
 - Follow the correct format and grammar
 - Prove over encryption algorithms, as AES-GCM/Chacha-Poly as AEAD
 - Prove statements
- Discussion and applications: why we might want this
 - Working over multiple HTTP requests
 - Working over browsers and DOM
 - Limits

Outline

The design of a complex/distributed system

- How to commit to TLS 1.3 encrypted data
- How to prove **in zkp** that said data has certain statements:
 - Follow the correct format and grammar
 - Prove over AES-GCM/ChaCha-Poly as AEAD
 - Prove statements
- Discussion and applications:
 - Working over multiple HTTP requests
 - Working over browsers and DOM
 - Limits

This lecture!
We can discuss future work
on the other ideas

Mise-en-scène

- We want to be able to prove *something while preserving privacy* (*)
 - To give the ability to users to access services or perform actions:
 - Attest humanity
 - Attest state
 - Attest attributes
- We cannot send that *something* in the clear → use *zero knowledge proofs*

In the web, we mostly rely on TLS for communication:

- Can we rely on it *to generate proofs*? How to prove *provenance*?
- Such channels commonly transmit trusted information about users behind clients such as proofs of age, social security statuses, and accepted purchase information

Mise-en-scène

In the web, we mostly rely on TLS for communication:

- Can we rely on it *to generate proofs*? How to prove *provenance*?
- Such channels commonly transmit trusted information about users behind clients such as proofs of age, social security statuses, and accepted purchase information

This is not the “only way”:

- JWTs, for instance, are used for authentication and authorization
 - We will see this in the next lecture
- Signed HTML messages

Mise-en-scène

- However, they follow the same “idea”:
 - Have an authenticated and non-maliciously crafted byte stream
 - Commit to a byte stream
 - Prove that is “well-formed”
 - Prove “facts” about the byte stream
- Examples: zkTLS, zkAuthorization, zkMiddleboxes, zkCompilation

Mise-en-scène

If we want to prove that the field “age” in a JSON-based byte stream (coming from TLS, for example) has a value greater than 1 (a proof of a statement):

- We leverage TLS: the channel where we are assured data is not “maliciously” formed
- How to commit to the transmitted data over TLS?
- How can we demonstrate that the data conforms to the JSON context-free grammar(*)?
- How can we demonstrate that the field “age” exists at the top level?
- How can we demonstrate that the value associated with “age” matches the grammar rule for integers?
- How can we demonstrate that the value of “age” satisfies a semantic constraint: While grammars handle structural validity, proving that the value of “age” is greater than 1 requires an additional semantic check beyond the grammar rules

Mise-en-scène

If we want to prove that the field “age” in a JSON-based byte stream (coming from TLS, for example) has a value greater than 1 (a proof of a statement):

```
{
  "age": 7,
  "user": {
    "id": "a91f2c13-55e2-4b5f-bc9e-1f920ae4d3fe",
    "profile": {
      "name": "Alice Example",
      "email": "alice@example.com",
      "preferences": {
        "language": "en",
        "notifications": true
      }
    }
  }
},
```

Mise-en-scène

If we want to prove that the field “age” in a JSON-based byte stream (coming from TLS, for example) has a value greater than 1 (a proof of a statement):

- We leverage TLS: the channel where we are assured data is not “maliciously” formed
- How to commit to the transmitted data over TLS?
- How can we demonstrate that the data conforms to the JSON context-free grammar(*)?
- How can we demonstrate that the field “age” exists at the top level?
- How can we demonstrate that the value associated with “age” matches the grammar rule for integers?
- How can we demonstrate that the value of “age” satisfies a semantic constraint: While grammars handle structural validity, proving that the value of “age” is greater than 1 requires an additional semantic check beyond the grammar rules

Designated-Commitment TLS (DCTLS)

(or *three-party handshake protocols* -3PS- or *zkTLS*(*))

- Modified TLS handshakes that allow **exporting statements** sent over the **TLS channel** to a **designated verifier**
 - Perform handshakes that secret-share private session data amongst a client and a verifier, and compute the handshake and record-layer phases in two-party computation (2PC), a form of MPC
- Examples:
 - DECO, TLSNotary/PageSigner, TownCrier, Garble-then-Prove, and Ianus
 - Zero-knowledge middleboxes
 - Multi-party TLS clients/servers: Oblivious TLS and MPCAuth

Designated-Commitment TLS (DCTLS)

They have limitations:

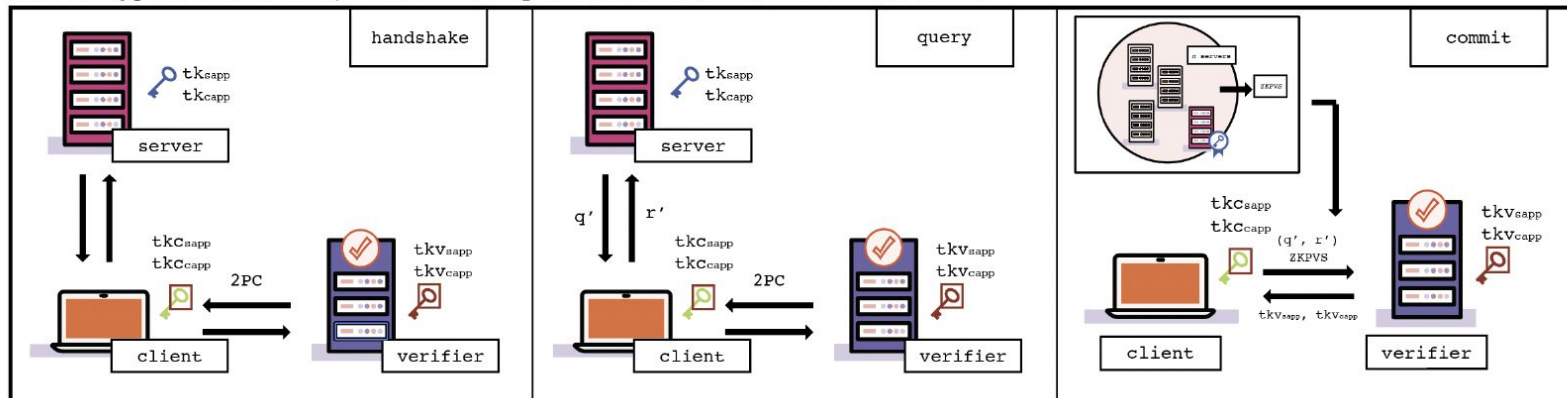
- No explicit/detailed support for TLS 1.3
 - It is not trivial to extend something from TLS 1.2 for TLS 1.3
- Lack of formal security analysis
- Efficiency concerns:
 - Unspecified usage of AES-GCM/ChachaPoly
- No client privacy: the identity of the server is revealed
- No open source implementations



- We introduce **DiStefano**:
<https://github.com/brave-experiments/distefano> ,
<https://www.ndss-symposium.org/wp-content/uploads/2025-1367-paper.pdf>

Designated-Commitment TLS (DCTLS)

Figure 1. An overview of the DiStefano protocol. In the **handshake** and **query** phases, the client performs the TLS 1.3 handshake and record-layer phases in conjunction with the verifier using 2PC to secret-share traffic keys and other session data for establishing a secure session with the server (secret-shared keys are represented with a square over the key). In the **commitment** phase, the client authenticates the server to the verifier using a zero-knowledge proof of valid TLS signatures (denoted by ZKPVS, see [Appendix C](#)), and commits to some encrypted session data, before receiving the verifier's secret TLS session shares.



Designated-Commitment TLS (DCTLS)

TLS 1.3:

- We saw this!
- Stronger security properties, the majority of handshake messages are encrypted
- Division between handshake keys and record-layer keys
- Handshake Layer:
 - How can we secret-share session data with a third party?
 - How can we assure that the data is committed to?
- Record Layer:
 - How can we create proofs over committed queries and responses?

```
Client      Server
static (Sig): pkS, skS

ClientHello:
rc ←$ {0,1}256, xc ←$ Zq
+ClientKeyShare: Xc ← gxc

ServerHello: rs ←$ {0,1}256, ys ←$ Zq
+ServerKeyShare: Ys ←$ gys
DHE ← Ysxc      DHE ← Xcys
HS ← HKDF.Extract(∅, DHE)
CHTS ← HKDF.Expand(HS, Label1 || H0)
SHTS ← HKDF.Expand(HS, Label2 || H0)
dHS ← HKDF.Expand(HS, Label3 || H1)
tkchs ← DeriveTK(CHTS)
tkshs ← DeriveTK(SHTS)

{+EncryptedExtensions }
{+CertificateRequest }*
{+ServerCertificate:}pkS
{+ServerCertificateVerify:}
Sig ← Sign(skS, Label7 || H3)
fkS ← HKDF.Expand(SHTS, Label4 || He)
{+ServerFinished:} SF ← HMAC(fkS, H4)
MS ← HKDF.Extract(dHS, ∅)
CATS ← HKDF.Expand(MS, Label5 || H2)
SATS ← HKDF.Expand(MS, Label6 || H2)
{+ClientCertificate:}*pkC
{+ClientCertificateVerify:}*
Sig ← Sign(skC, Label8 || H5)
fkC ← HKDF.Expand(CHTS, Label4 || He)
{+ClientFinished:} CF ← HMAC(fkC, H6)
tkcapp ← DeriveTK(CATS)
tksapp ← DeriveTK(SATS)
abort if Verify(pkC, Label8 || H5, Sig) ≠ 1
abort if CF ≠ HMAC(fkC, H6)
```

Designated-Commitment TLS (DCTLS)

In practice:

- Secret share session keys between **Client and Verifier**:
 - Verifier never has a “complete view” of the shares nor plaintext
 - Verifier commits to the encrypted data: both handshake-layer (from *ServerEncryptedExtensions* message onwards) and record-layer messages
- Commit and reveal/prove system:
 - Commit to handshake messages
 - Commit to query/responses at the record layer

Note: other systems do not need the “verifier” entity:

- Here we attest “online” interactions (*)

Designated-Commitment TLS (DCTLS)

1. Handshake phase:

Server learns the same secret session parameters as in standard TLS 1.3, while the client and verifier learn shares of the session parameters: engage in the core TLS 1.3 protocol using a series of 2PC functionalities.

2. Query phase:

The client sends queries to the server with help from the verifier. Since the session keys are secret-shared, the client and verifier jointly compute the encryptions of these queries in 2PC. The response can then be decrypted using a similar procedure.

3. Commitment phase:

After querying and receiving responses, the client commits to the session by forwarding the ciphertexts to the verifier, and receives the verifier's session key shares in exchange. The client can verify the integrity of responses and later prove statements about them.

Designated-Commitment TLS (DCTLS): *DiStefano*

1. Handshake phase:

Server learns the same secret session parameters as in standard TLS 1.3, while the client and verifier learn shares of the session parameters: engage in the core TLS 1.3 protocol using a series of 2PC functionalities.

2. Query phase:

The client sends queries to the server with help from the verifier. Since the session keys are secret-shared, the client and verifier jointly compute the encryptions of these queries in 2PC. The response can then be decrypted using a similar procedure.

3. Commitment phase:

After querying and receiving responses, the client commits to the session by forwarding the ciphertexts to the verifier, and receives the verifier's session key shares in exchange. The client can verify the integrity of responses and later prove statements about them.

DCTLS: *DiStefano*

Handshake phase: preprocessing phase

- Client and Verifier act as a single entity
- Server remains the same
- We “reverse” the traditional flow of the handshake so that a key share is computed in advance

Verifier

ClientHello:

$z_v \leftarrow \mathbb{Z}_q, Z_v \leftarrow g^{z_v}$

+ClientKeyShare: $SSK \leftarrow Z_v + X_c$

Client

ClientHello:

$x_c \leftarrow \mathbb{Z}_q, X_c \leftarrow g^{x_c}$

+ClientKeyShare: $SSK \leftarrow Z_v + X_c$

Server

static (Sig): pk_S, sk_S

DCTLS: *DiStefano*

Handshake phase: key agreement

- Client and Verifier have additive shares: they operate on the “x” coordinate
 - Operate on the field instead of the curve
- A binary-garbled circuit for the 2PC functionality:
 - Handshake keys derivation
 - Encryption and Decryption (for some parts of it)

$$\begin{aligned} \text{HS}^v \oplus \text{HS}^c &\leftarrow \text{HKDF}.\text{Extract}(\emptyset, t_v + t_c) \\ \text{CHTS}^v \oplus \text{CHTS}^c &\leftarrow \text{HKDF}.\text{Expand}(\text{HS}^v \oplus \text{HS}^c, \text{Label}_1 \parallel H_0) \\ \text{SHTS}^v \oplus \text{SHTS}^c &\leftarrow \text{HKDF}.\text{Expand}(\text{HS}^v \oplus \text{HS}^c, \text{Label}_2 \parallel H_0) \\ \text{dHS}^v \oplus \text{dHS}^c &\leftarrow \text{HKDF}.\text{Expand}(\text{HS}^v \oplus \text{HS}^c, \text{Label}_3 \parallel H_1) \\ \text{tk}_{chs}^v \oplus \text{tk}_{chs}^c &\leftarrow \text{DeriveTK}(\text{CHTS}^v \oplus \text{CHTS}^c) \\ \text{tk}_{shs}^v \oplus \text{tk}_{shs}^c &\leftarrow \text{DeriveTK}(\text{SHTS}^v \oplus \text{SHTS}^c) \end{aligned}$$
$$\begin{aligned} \text{DHE} &\leftarrow \text{SSK}^{y_s} \\ \text{HS} &\leftarrow \text{HKDF}.\text{Extract}(\emptyset, \text{DHE}) \\ \text{CHTS} &\leftarrow \text{HKDF}.\text{Expand}(\text{HS}, \text{Label}_1 \parallel H_0) \\ \text{SHTS} &\leftarrow \text{HKDF}.\text{Expand}(\text{HS}, \text{Label}_2 \parallel H_0) \\ \text{dHS} &\leftarrow \text{HKDF}.\text{Expand}(\text{HS}, \text{Label}_3 \parallel H_1) \\ \text{tk}_{chs} &\leftarrow \text{DeriveTK}(\text{CHTS}) \\ \text{tk}_{shs} &\leftarrow \text{DeriveTK}(\text{SHTS}) \\ \{ &+ \text{EncryptedExtensions} \} \end{aligned}$$

DCTLS: *DiStefano*

Handshake phase: authentication and integrity

```

    {+ServerCertificate:}pkS
    {+ServerCertificateVerify:}
    SigS ← Sign(skS, Label7 || H3)
    fkS ← HKDF.Expend(SHTS, Label4 || Hε)
    {+ServerFinished:} SF ← HMAC(fkS, H4)

    Reveal SHTSv to Client
    Forward encrypted {EE}, ..., {SF} to Verifier

    Derive tkchs using SHTSv
    abort if Verify(pkS, Label7 || H3, SigS) ≠ 1
    abort if SF ≠ HMAC(fkS, H4)
    Forward SF, σ ← Π.Prove(R, SigS, Label7 || H3) to Verifier
    Reveal fkS to Verifier
    Forward H4, H3 and H2 to Verifier

    abort if SF ≠ HMAC(fkS, H4) or 0 ← Π.Verify(R, σ, Label7 || H3)
    MSv ⊕ MSc ← HKDF.Extract(dHSv ⊕ dHSc, ∅)
    CATSv ⊕ CATSc ← HKDF.Expend(MSv ⊕ MSc, Label5 || H2)
    SATSv ⊕ SATSc ← HKDF.Expend(MSv ⊕ MSc, Label6 || H2)
    tkcappv ⊕ tkcappc ← DeriveTK(CATSv ⊕ CATSc)
    tksappv ⊕ tksappc ← DeriveTK(SATSv ⊕ SATSc)
    MS ← HKDF.Extract(dHS, 0)
    CATS ← HKDF.Expend(MS, Label5 || H2)
    SATS ← HKDF.Expend(MS, Label6 || H2)
    tkcapp ← DeriveTK(CATS)
    tksapp ← DeriveTK(SATS)

    {+ClientCertificate:}*pkC
    {+ClientCertificateVerify:}*
    SigC ← Sign(skC, Label8 || H5)

    Reveal CHTSv to Client
    fkC ← HKDF.Expend(CHTSv ⊕ CHTSc, Label4 || Hε)
    fkC ← HKDF.Expend(CHTS, Label4 || Hε)
    {+ClientFinished:} CF ← HMAC(fkC, H6)
    abort if Verify(pkC, Label8 || H5, SigC) ≠ 1
    abort if CF ≠ HMAC(fkC, H6)

```

DCTLS: DiStefano

Handshake phase: authentication and integrity

```
fkS ← HKDF.Expend(SHTSv ⊕ SHTSc, Label4 || Hε)
fkS ← HKDF.Expend(SHTS, Label4 || Hε)
{+ServerCertificate:}pkS
{+ServerCertificateVerify:}
SigS ← Sign(skS, Label7 || H3)
{+ServerFinished:} SF ← HMAC(fkS, H4)
Forward encrypted {EE}, ..., {SF} to Verifier
Reveal SHTSv to Client
Derive tkchs using SHTSv
abort if Verify(pkS, Label7 || H3, SigS) ≠ 1
abort if SF ≠ HMAC(fkS, H4)
Forward SF, σ ← Π.Prove(R, SigS, Label7 || H3) to Verifier
Reveal Sig to Verifier
Forward H4, H3 and H2 to Verifier
abort if SF ≠ HMAC(fkS, H4) or 0 ← Π.Verify(R, σ, Label7 || H3)
MSv ⊕ MSc ← HKDF.Extract(dHSv ⊕ dHSc, ∅)
MS ← HKDF.Extract(dHS, 0)
CATSv ⊕ CATSc ← HKDF.Expend(MSv ⊕ MSc, Label5 || H2)
CATS ← HKDF.Expend(MS, Label5 || H2)
SATSv ⊕ SATSc ← HKDF.Expend(MSv ⊕ MSc, Label6 || H2)
SATS ← HKDF.Expend(MS, Label6 || H2)
tkcappv ⊕ tkcappc ← DeriveTK(CATSv ⊕ CATSc)
tkcapp ← DeriveTK(CATS)
tksappv ⊕ tksappc ← DeriveTK(SATSv ⊕ SATSc)
tksapp ← DeriveTK(SATS)
{+ClientCertificate:}pkC
{+ClientCertificateVerify:}*
SigC ← Sign(skC, Label8 || H5)
Reveal CHTSv to Client
fkC ← HKDF.Expend(CHTSv ⊕ CHTSc, Label4 || Hε)
fkC ← HKDF.Expend(CHTS, Label4 || Hε)
{+ClientFinished:} CF ← HMAC(fkC, H6)
abort if Verify(pkC, Label8 || H5, SigC) ≠ 1
abort if CF ≠ HMAC(fkC, H6)
```

Prove that the signature is valid in regards to a public set of keys, without revealing which one: ZKPVS

Sofia Celi, Shai Levin, Joe Rowell, CDLS:
Proving Knowledge of Committed Discrete Logarithms with Soundness, International Conference on Cryptology in Africa 2024 (AfricaCrypt2024):

<https://eprint.iacr.org/2023/1595>

DCTLS: *DiStefano* → encryption/decryption

TLS 1.3: **AES-GCM** and ChachaPoly in a AEAD setting (*focusing in AES-GCM*):

- Remember: AES is a block cipher with an authenticity tag
- Both Client and Verifier learn all AES-GCM ciphertext blocks $C_i = M_i \oplus AES.Enc(k, IV + i)$ produced by the server, where the session key k is secret-shared
- **Problem:** the client has to create a commitment [1] to the received ciphertext blocks without revealing their key share:
 - Non-committing attack: $C_i = M'_i \oplus AES.Enc(k', IV + i)$
 - Validate the tag in 2PC: (C_i, τ)
 - Commit to masks (b_i) , and send commitments to them and the key (in 2PC AES-Decrypt):
 - $e_i = AES.Enc(k, IV + i)$ and $E_i = e_i + b_i$
 - (Verifier reveals their k)
 - $M_i = C_i \oplus AES.Enc(k_c + k_v, IV_c + i)$ $i \in [n]$ to the Client
 - $(E_i = AES.Enc(k_c + k_v, IV_c + i) \oplus b_i)$ $i \in [n]$ to the Verifier → commitment
 - Client can forward now blocks e_i

[1] P. Grubbs, J. Lu, and T. Ristenpart, *Message franking via committing authenticated encryption* in CRYPTO 2017

DCTLS: *DiStefano* → encryption/decryption

Algorithm 1 2PC-AES-GCM.Enc [7]

Public: associated data A , block count n .

Input ($\mathcal{C}$): k_C , IV_C , message M , GHASH power shares $\{h_C^i\}_{i \in [n]}$.

Input ($\mathcal{V}$): k_V , IV_V , GHASH power shares $\{h_V^i\}_{i \in [n]}$.

Output ($\mathcal{C}$): (C, τ) . **Output** ($\mathcal{V}$): C .

Preconditions: $k = k_C \oplus k_V$; neither party has used IV_C or IV_V for encryption before.

```
1: if  $IV_C \neq IV_V$  then
2:   abort                                ▷ the nonces must agree
3: end if
4:  $IV \leftarrow IV_C$ 
5: decompose  $M$  as  $M_1 \parallel \dots \parallel M_n$ 
6: for all  $i \in [n]$  do                      ▷ jointly
7:    $C_i \leftarrow \text{AES.Enc}(k_C \oplus k_V, IV \parallel \langle i \rangle) \oplus M_i$ 
8: end for
9:  $C \leftarrow C_1 \parallel \dots \parallel C_n$ 
10:  $X \leftarrow A \parallel C \parallel \text{len}(A) \parallel \text{len}(C)$       ▷ GHASH input
11:  $\tau_C \leftarrow P_X(\{h_C^i\}_{i \in [n]})$                     ▷ locally, by  $\mathcal{C}$ 
12:  $\tau_V \leftarrow P_X(\{h_V^i\}_{i \in [n]})$                     ▷ locally, by  $\mathcal{V}$ 
13:  $\tau \leftarrow \tau_C \oplus \tau_V \oplus \text{AES.Enc}(k_C \oplus k_V, IV \parallel \langle 0 \rangle)$   ▷  $P_X$  is
    linear in the powers of  $H$ 
14: output  $(C, \tau)$  to  $\mathcal{C}$  and  $C$  to  $\mathcal{V}$ 
```

TLS 1.3: AES-GCM, encryption:

1. Encrypt in 2PC
 2. Locally perform a bunch of the GHASH computation ahead of time, and only the last part in MPC
- Why do we need to compute this in MPC?

DCTLS: *DiStefano* → encryption/decryption

Algorithm 6 Optimised Keystream 2PC-AES-GCM.Dec

Setup: GHASH power shares $\{h_C^j\}_{j \in [n]}$, $\{h_{\mathcal{V}}^j\}_{j \in [n]}$.

Public: associated data A , n , record $C = C_1 \parallel \dots \parallel C_n$, tag τ , batched width $B = 16$.

Input (C): key share k_C , IV_C , mask seed s .

Input ($\mathcal{V}$): key share $k_{\mathcal{V}}$, $IV_{\mathcal{V}}$, commitment d_s , challenge t' .

Output (C): M , or $\perp$.

Output ($\mathcal{V}$): $E_1, \dots, E_n$, or $\perp$.

Precondition: IV_C , $IV_{\mathcal{V}}$ not previously used for encryption.

// MAC (integrity)

```
1: if  $IV_C \neq IV_{\mathcal{V}}$  then
2:   return Error                                ▷ The IVs must match.
3: end if
4:  $\tau'_C \leftarrow P_{A \parallel C \parallel \text{len}(A) \parallel \text{len}(C)}(\{h_C^i\})$ 
5:  $\tau'_V \leftarrow P_{A \parallel C \parallel \text{len}(A) \parallel \text{len}(C)}(\{h_V^i\})$ 
6:  $\tau' = \tau'_C \oplus \tau'_V \oplus \text{AES.Enc}(k_C + k_V, IV_C)$ 
7: if  $\tau' \neq \tau$  then
8:   return Error                                ▷ Invalid tag
9: end if
```

// Decryption (confidentiality)

```
10: for  $\beta = 0$  to  $\lceil n/B \rceil - 1$  do
11:   if  $0 \leftarrow \Gamma'.\text{Open}(d; t', s)$  then
12:     return Error                                ▷ Seed commitment check failed.
13:   end if
14:   for  $j = 0$  to  $B - 1$  do
15:      $\mathfrak{t} \leftarrow \beta \cdot B + j$ 
16:     if  $\mathfrak{t} < n$  then
17:        $\text{ctr}_{\mathfrak{t}} \leftarrow IV_C \parallel (\mathfrak{t} + 2)$     ▷ Counters 0, 1 reserved
18:       for  $J_0$ .
19:         else
20:            $\text{ctr}_{\mathfrak{t}} \leftarrow 0x80000000 \parallel \mathfrak{t}$     ▷ Padding slot
21:         end if
22:          $b_{\mathfrak{t}} \leftarrow \text{AES.Enc}(s, \text{ctr}_{\mathfrak{t}})$     ▷ Mask derived in-circuit
23:          $E_{\mathfrak{t}} \leftarrow \text{AES.Enc}(k_C + k_V, \text{ctr}_{\mathfrak{t}}) \oplus b_{\mathfrak{t}}$ 
24:       end for
25:       for  $\mathfrak{t} = \beta \cdot B$  to  $\min((\beta + 1)B, n) - 1$  do
26:          $M_{\mathfrak{t}} \leftarrow C_{\mathfrak{t}} \oplus E_{\mathfrak{t}} \oplus \text{AES.Enc}(s, \text{ctr}_{\mathfrak{t}})$ 
27:       end for
28:     end for
29:   return  $(M_{\mathfrak{t}})_{\mathfrak{t} \in [n]}$  to  $C$ 
30:   return  $(E_{\mathfrak{t}})_{\mathfrak{t} \in [\beta \cdot B, \min((\beta + 1) \cdot B, n)]}$  to  $\mathcal{V}$ 
```

DCTLS: *DiStefano* → encryption/decryption

Algorithm 6 Optimised Keystream 2PC-AES-GCM.Dec

Setup: GHASH power shares $\{h_C^j\}_{j \in [n]}$, $\{h_{\mathcal{V}}^j\}_{j \in [n]}$.

Public: associated data A , n , record $C = C_1 \parallel \dots \parallel C_n$, tag τ , batched width $B = 16$.

Input (C): key share k_C , IV_C , mask seed s .

Input ($\mathcal{V}$): key share $k_{\mathcal{V}}$, $IV_{\mathcal{V}}$, commitment d_s , challenge t' .

Output (C): M , or $\perp$.

Output ($\mathcal{V}$): $E_1, \dots, E_n$, or $\perp$.

Precondition: IV_C , $IV_{\mathcal{V}}$ not previously used for encryption.

// MAC (integrity)

```
1: if  $IV_C \neq IV_{\mathcal{V}}$  then
2:   return Error                                ▷ The IVs must match.
3: end if
4:  $\tau'_C \leftarrow P_{A \parallel C \parallel \text{len}(A) \parallel \text{len}(C)}(\{h_C^i\})$ 
5:  $\tau'_V \leftarrow P_{A \parallel C \parallel \text{len}(A) \parallel \text{len}(C)}(\{h_V^i\})$ 
6:  $\tau' = \tau'_C \oplus \tau'_V \oplus \text{AES.Enc}(k_C + k_V, IV_C)$ 
7: if  $\tau' \neq \tau$  then
8:   return Error                                ▷ Invalid tag
9: end if
```

// Decryption (confidentiality)

```
10: for  $\beta = 0$  to  $\lceil n/B \rceil - 1$  do
11:   if  $0 \leftarrow \Gamma'.\text{Open}(d; t', s)$  then
12:     return Error                                ▷ Seed commitment check failed.
13:   end if
14:   for  $j = 0$  to  $B - 1$  do
15:      $\mathfrak{t} \leftarrow \beta \cdot B + j$ 
16:     if  $\mathfrak{t} < n$  then
17:        $\text{ctr}_{\mathfrak{t}} \leftarrow IV_C \parallel (\mathfrak{t} + 2)$     ▷ Counters 0, 1 reserved
        for  $J_0$ .
18:       else
19:          $\text{ctr}_{\mathfrak{t}} \leftarrow 0x80000000 \parallel \mathfrak{t}$     ▷ Padding slot
20:       end if
21:        $b_{\mathfrak{t}} \leftarrow \text{AES.Enc}(s, \text{ctr}_{\mathfrak{t}})$     ▷ Mask derived in-circuit
22:        $E_{\mathfrak{t}} \leftarrow \text{AES.Enc}(k_C + k_V, \text{ctr}_{\mathfrak{t}}) \oplus b_{\mathfrak{t}}$ 
23:     end for
24:     for  $\mathfrak{t} = \beta \cdot B$  to  $\min((\beta + 1) \cdot B, n) - 1$  do
25:        $M_{\mathfrak{t}} \leftarrow C_{\mathfrak{t}} \oplus E_{\mathfrak{t}} \oplus \text{AES.Enc}(s, \text{ctr}_{\mathfrak{t}})$ 
26:     end for
27:   end for
28: return  $(M_{\mathfrak{t}})_{\mathfrak{t} \in [n]}$  to  $C$ 
29: return  $(E_{\mathfrak{t}})_{\mathfrak{t} \in [\beta \cdot B, \min((\beta + 1) \cdot B, n)]}$  to  $\mathcal{V}$ 
```

DCTLS: *DiStefano* → encryption/decryption

TLS 1.3: **AES-GCM, decryption:**

1. Locally perform a bunch of the GHash computation ahead of time, and only the last part in MPC
 2. Generate a hiding and binding commitment to the ciphertext and key
- Why do we need to generate a commitment to the key?
 - What is this algorithm linear in?

DCTLS: *DiStefano* → encryption/decryption

- Why do we need to generate a commitment to the key?
- What is this algorithm linear in?
 - Linear in the ciphertext blocks
 - It is expensive!
 - For 26653 B (the size of an image), it takes 30.73 minutes to decrypt

DCTLS: *DiStefano*

Implementation

- Implementation in C++ in BoringSSL:
 - 14k lines of code
 - The library used in the majority of browsers
- Evaluated in a LAN and WAN setting
 - LAN: we use a consumer-grade device (a Macbook air M1 with 8GB of RAM) for the Client, and a server-grade device (an Intel Xeon Gold 6138 with 32GB of RAM) for the Verifier and Server. All communication used in TLS 1.3 was carried out using a single thread over a 1 Gbps network with a latency of around *16 ms*.
 - WAN: we use two AWS EC2 “t2.2xlarge” machines: one for the Client located in Spain, and one for the Verifier and the Server in Ohio, USA, where the median latency is estimated at *100 ms*.
 - Timings and bandwidth measurements are computed as the mean of 50 samples, and are represented in milliseconds and mebibytes, respectively (1 MiB is 220 bytes).

DCTLS: *DiStefano*

Implementation

- < 1 s and ≤ 80 KiB to execute the *online phase* (just the handshake):
 - Less than TLS handshake timeout times which, while configurable, are typically between 10 and 20 seconds

Table 4. E2E TIMINGS (MS) AND BANDWIDTH (MiB) FOR DCTLS IN WAN SETTINGS. ALL TIMES ARE REPORTED IN MS.

Process	WAN-Ohio	WAN-London	WAN-Seoul
<i>Offline costs</i>			
<i>C/S</i> Key Share	102.4682	12.2567	252.1662
<i>C/V</i> execute ECtF	112.6829	2.5680	285.9367
Circuit Preprocessing	8958.3445	6236.5134	10417.6417
<i>Online costs</i>			
<i>S</i> sends cert.	125.9581	2.3673	12.2567
Derive traffic secrets	130.9039	43.6089	273.2959
Derive GCM shares	494.3445	149.2039	872.4691

Table 3. E2E TIMINGS (MS) AND BANDWIDTH (MiB) FOR DCTLS IN LAN SETTINGS (WITH LATENCY ≈ 16 ms).

Process	LAN (ms)	Bandwidth (MiB)
<i>Offline costs</i>		
<i>C/S</i> Key Share	1.3167	6.67572e-05
<i>C/V</i> execute ECtF	0.008083	9.53674e-07
Circuit Preprocessing	6280.08	220.484
<i>Online costs</i>		
<i>S</i> sends cert.	0.011375	3.14713e-05
Derive traffic secrets	33.1389	0.0276108
Derive GCM shares	136.573	0.0488291

Designated-Commitment TLS (DCTLS)

1. Handshake phase:

Server learns the same secret session parameters as in standard TLS 1.3, while the client and verifier learn shares of the session parameters: engage in the core TLS 1.3 protocol using a series of 2PC functionalities.

2. Query phase:

The client sends queries to the server with help from the verifier. Since the session keys are secret-shared, the client and verifier jointly compute the encryptions of these queries in 2PC. The response can then be decrypted using a similar procedure.

3. Commitment phase:

After querying and receiving responses, the client commits to the session by forwarding the ciphertexts to the verifier, and receives the verifier's session key shares in exchange. The client can verify the integrity of responses and later prove statements about them.

DCTLS: *DiStefano* → queries

Record phase: commitments for proofs over transmitted data

- *DiStefano*:
 - Ability to create commitments to AES-GCM encrypted data transmitted over TLS 1.3:
 - Allows for *selective opening*: we consider this intrusive and hence will not use it
 - Can easily be extended to ChachaPoly
- TLS 1.3: works mostly over HTTP with HTML/JSON format
- Other formats like XML, binary data, or proprietary protocols can also be transmitted securely over TLS
- HTTP method: query and response

DCTLS: *DiStefano* → queries

```
{
  "name": "DiStefano Protocol",
  "version": "1.0",
  "description": "A DCTLS protocol over TLS 1.3.",
  "age": 2,
  "features": {
    "security": "Proven fully secure",
    "privacy": "Maintains client privacy",
    "efficiency": "Highly efficient"
  },
  "performance": {
    "runtimeImpact": "Proportional to latency",
    "gcmShareDerivationTime": "Less than a second",
    "totalOnlineCost": "Below typical TLS handshake timeout"
  },
  "latencyImpact": "Sublinear"
}
```

DCTLS: *DiStefano* → queries

```
{
  "name": "DiStefano Protocol",
  "version": "1.0",
  "description": "A DCTLS protocol over TLS 1.3.",
  "age": 2,
  "features": {
    "security": "Proven fully secure",
    "privacy": "Maintains client privacy",
    "efficiency": "Highly efficient"
  },
  "performance": {
    "runtimeImpact": "Proportional to latency",
    "gcmShareDerivationTime": "Less than a second",
    "totalOnlineCost": "Below typical TLS handshake timeout"
  },
  "latencyImpact": "Sublinear"
}
```

- The return of an *HTTP GET request*
- How do we prove that this field exist in the provided data encrypted via AES with a JSON format?
- Important that malformed JSON is not validated as correct

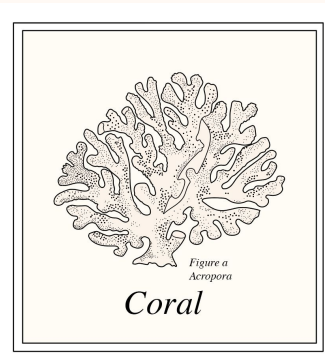
Mise-en-scene

If we want to prove that the field “age” in a JSON-based byte stream (coming from TLS, for example) has a value greater than 1 (a proof of a statement):

- We leverage TLS: the channel where we are assured data is not “maliciously” formed
- How to commit to the transmitted data over TLS?
- How can we demonstrate that the data conforms to the JSON context-free grammar(*)?
- How can we demonstrate that the field “age” exists at the top level?
- How can we demonstrate that the value associated with “age” matches the grammar rule for integers?
- How can we demonstrate that the value of “age” satisfies a semantic constraint: While grammars handle structural validity, proving that the value of “age” is greater than 1 requires an additional semantic check beyond the grammar rules

Coral: Fast Succinct Non-Interactive Zero-Knowledge CFG Proofs

- Why do we need to check that the byte stream is conformant to a public grammar?
 - Because requests and responses can trick us
 - A malicious client can use error responses as valid
- One can do this easily:
 - Put the parsing functionality as a R1CS instance and prove in ZKP
 - Expensive. Can we do better?
- Coral: <https://eprint.iacr.org/2025/1420.pdf>



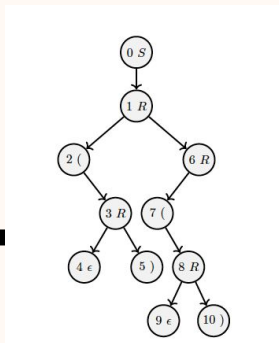
```
HTTP 400 Bad Request.  
The Email "'email': 'admin@domain.com'" is invalid.
```

Coral: Fast Succinct Non-Interactive Zero-Knowledge CFG Proofs

- We look at Context Free Grammars
- Cheaper: checking that an answer is correct rather than computing an answer (linear instead of quadratic for our case)
- We arithmetize an efficient **NP checker** (proposed solution can be checked efficiently) that takes as input the byte stream, a grammar (any CFG), the parsed object, and auxiliary hints:
 - a. Verifies that the parsed object results from parsing the byte stream according to the grammar
- Goal: prove
 - a. A committed byte stream C_b commits to an object B .
 - b. The object B when parsed (its parsed tree) according to a public grammar is committed to C_t .
 - c. B and T correspond to each other.

Coral: Fast Succinct Non-Interactive Zero-Knowledge CFG Proofs

- We arithmetize an efficient **NP checker** that takes as input the byte stream, a grammar, the parsed object, and auxiliary hints:
 - The parsed object is represented as a parse tree
 - Convert it to a left-child right-sibling (LCRS) tree:
 - Bound the number of leaves
 - Build the R1CS instances:
 - Walk the tree (in Depth-First Search) to check each node conforms to the grammar
 - The prover recursively proves each instance using a folding scheme, showing that it used a Depth-First Search order



$S \rightarrow \text{non-terminal}$
 $\text{Leaf} \rightarrow \text{either a symbol or empty}$

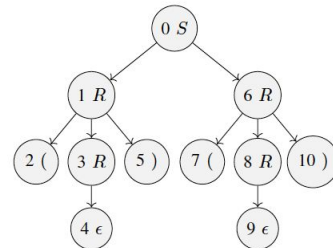
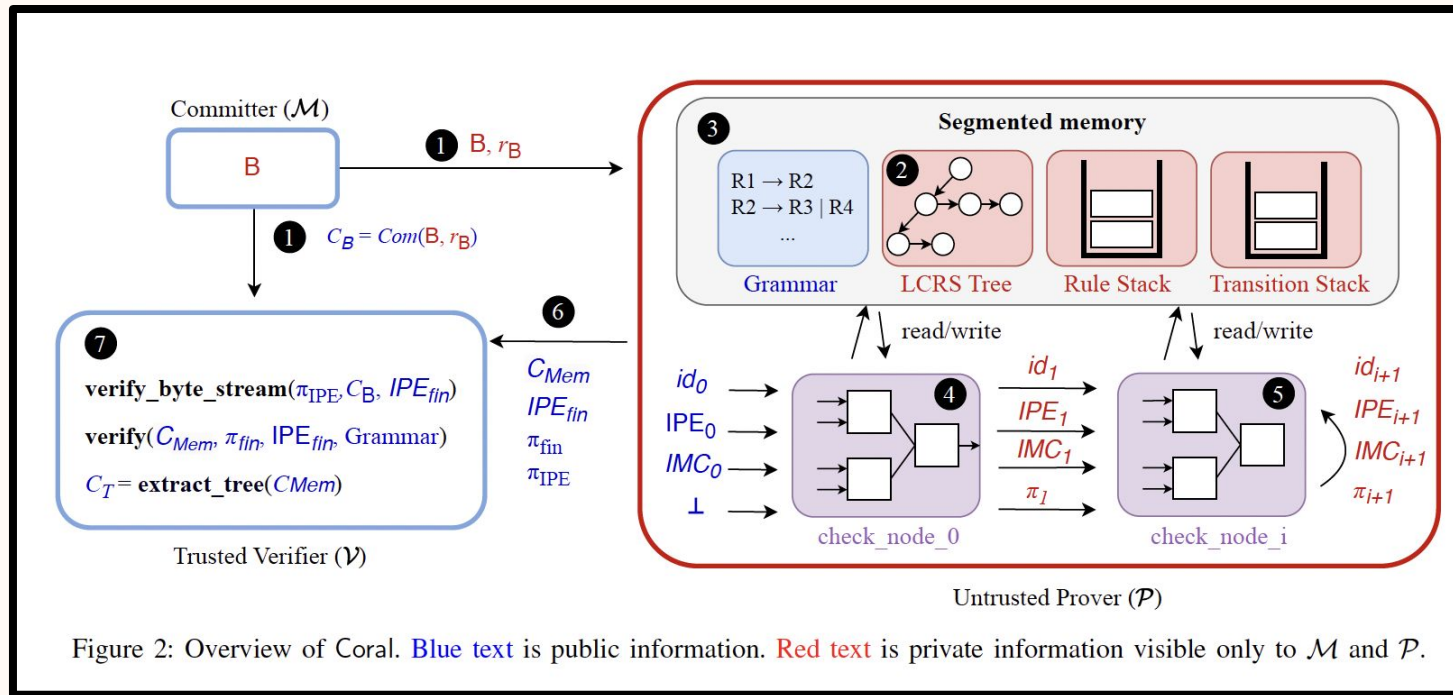


Figure 1: Parse tree for the string “ $()()$ ” and the grammar $S \rightarrow (S) \mid SS \mid \epsilon$. We use ϵ to denote the empty string.

Coral: Fast Succinct Non-Interactive Zero-Knowledge CFG Proofs



Coral: Fast Succinct Non-Interactive Zero-Knowledge CFG Proofs

How?

1. We have a commitment to a byte stream C_B and the byte stream B
2. Create the parse tree for B in regards to public grammar G
 - a. Convert to an LCRS tree T , where each node has a unique id
3. Use a segmented memory to keep track of:
 - a. Public grammar G
 - b. Private LCRS tree T
 - c. Private stacks for track keeping: rules and transitions
4. Commit to T and private stacks: C_{mem} . Use an IVC
5. Run the *check_node* (next slides) function for each node, which increments the id and outputs:
 - a. An incremental polynomial that keeps tracks of the leaves visited (IPE)
 - b. An incremental commitment to memory operations (IMC)
 - c. A proof for the node: π

Coral: Fast Succinct Non-Interactive Zero-Knowledge CFG Proofs

How?

5. Run the *check_node* (next slides) function for each node, which increments the *id* and outputs:
 - a. An incremental polynomial that keeps tracks of the leaves visited (IPE)
 - b. An incremental commitment to memory operations (IMC)
 - c. A proof for the node: π
6. Make public the final values of the recursive function: IPE_fin (and a proof of it π_{IPE}), IMC_fin, and π_{fin}
7. Check that:
 - a. IPE_fin and C_B commit to the same byte stream
 - b. Check π_{fin} with C_mem and IMC_fin

```
Node {  
  id: field,  
  parent_id: field,  
  l_child_id: field,  
  r_sibling_id: field,  
  is_leaf: bool,  
  symbol: field,  
}
```

Coral: Fast Succinct Non-Interactive Zero-Knowledge CFG Proofs

```
check_node(field id, field ipe,
           field[] rule_stack, field[] tran_stack,
           field r1_B, field[] proposed_sym) {

    Node cur = get_node(id); // ROM access

    ipe = if (id == 0) {
        accumulate(ipe, r1_B) // poly eval
    } else {
        // validate prior proposed rules
        field (symbol, rule_end) = rule_stack.pop();
        assert(cur.symbol == symbol);
        assert((cur.r_sibling_id==NULL) == rule_end);

        // accumulate poly eval if it's a leaf
        if cur.is_leaf {
            accumulate(ipe, symbol)
        } else { ipe }
    };

    if !cur.is_leaf {
        // confirm exists in grammar (ROM access)
        assert(lookup(cur.symbol, proposed_sym));

        // number of nonzero symbols in proposal
        field n_sym = proposed_sym.length();

        for i in 1..n_symbols {
            rule_stack.push(proposed_sym[n_sym-i], i==1);
        }
    }

    if cur.r_sibling_id != NULL {
        tran_stack.push(cur.r_sibling_id);
    }

    field next_id = if cur.is_leaf {
        tran_stack.pop()
    } else { cur.l_child_id };

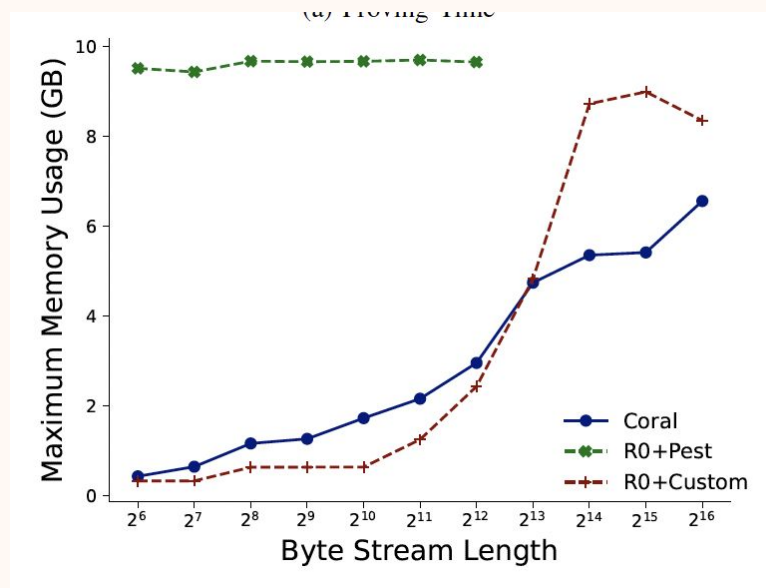
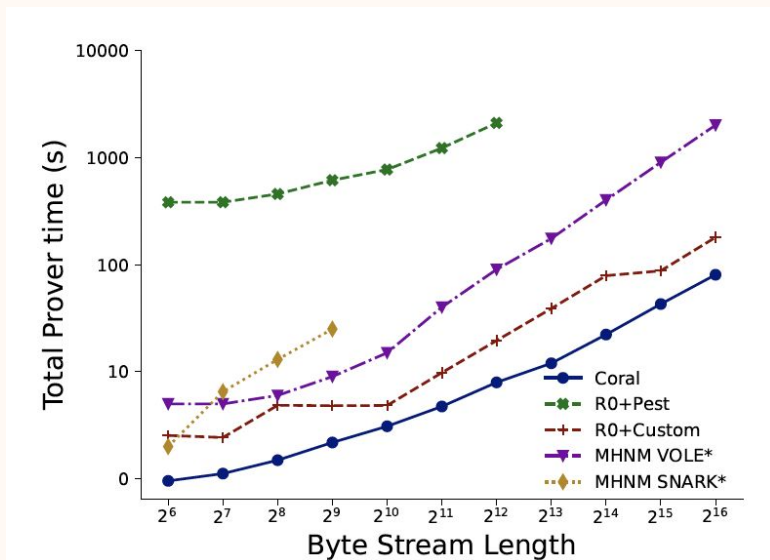
    assert(id + 1 == next_id);

    return (next_id, ipe, rule_stack, tran_stack);
}
```

To check that it corresponds to the byte stream

To check just the rule

Coral: Fast Succinct Non-Interactive Zero-Knowledge CFG Proofs



Coral: Limits and more

- We can check conformance to a grammar, but:
 - We cannot check uniqueness
 - We cannot check that a field exists at the root, for instance
- However, the *check_node* function can allow for this:
 - Keep track at memory
 - Future work
- Why we need so?
 - Attest that the field “age” is the only one that shows up
 - Attest that the field “age” is at the second level of a JSON document

```
{  
  "user": {  
    "age": 7  
  },  
}
```

Attacks

- Many systems do not check grammar conformance
- One can break them via:
 - Duplicate keys
 - Non-standard extensions
 - Different numeric encodings / strings vs numbers
 - Trailing commas / relaxed commas

```
"user": {  
  "age": 7,           // first age (verifier may keep this)  
  "name": "Alice",  
  "details": {  
    "a\u0067e": "19",    // second age, but key encoded as "age"  
    "preferences": {  
      "timezone": "UTC"  
    }  
  }  
},  
  
// duplicate key at top level  
"user": {  
  "Age": 0000019,    // third age; type ambiguity + weird numeric form  
  "profile": {  
    "age": 99,        // fourth age, parser-dependent if overwritten  
    "country": "US"  
  }  
},
```

Attacks

- There is an assumption that servers will “serve” well-formed responses, because they are “good providers”
 - But! this is incorrect: the majority of websites have malformed data
 - The browser fixes this on-the-fly when constructing the DOM
 - Servers “just work” because browsers have decades of robust recovery logic.
 - However, if we attest in ZKP that an encrypted response from a server is conformant with a grammar:
 - Does it correspond to what will show to user after DOM construction?
 - This also ignores the possibility of injecting data

Putting it all together

The design of a complex/distributed system

- *DiStefano*: provides already commitments (via AEAD) to encrypted blocks
- Prove over the AES/ChachaPoly blocks/stream:
 - That the JSON is well-formed: *Coral*
 - That the key exists and it is in the correct “path”
 - That the value associated with the key has a property:
 - is an integer
 - is bigger than

Putting it all together

The design of a complex/distributed system

- *DiStefano*: provides already comm
- Prove over the AES/ChachaPoly b
 - That the JSON is well-formed
 - That the key exists and it is i
 - That the value associated with
 - is an integer
 - is bigger than

Upcoming future work!

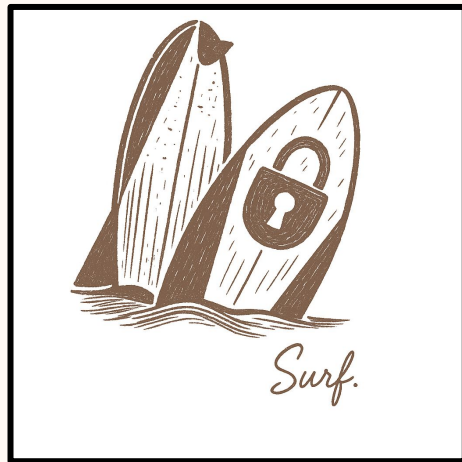
Notice that:

- We don't need to "re-commit"
- We can leverage the block structure of AES

Extra ideas

With DiStefano we have 1 fly of commitments prior to revealing the key: can we do for several:

- HTTP request and responses are several: GET, POST, etc
 - Interaction is based on user interaction
- Enter *Surf*: give the ability to prove in an interactive manner



However...

- ZKPs often pitched as a privacy-preserving solution to age or eligibility checks.
 - Security & soundness fragility: some systems don't satisfy full zero-knowledge or soundness definitions.
 - Privacy leakage via range proofs & session linkage: e.g., proving age in a narrow range may leak exact age over time.
 - Semantic & parsing mismatch risk: assuming well-formed input (e.g., JSON or TLS transcript) can be exploited if input is malformed.
 - Centralisation & exclusion issues: relying on a small set of credential-issuers can create gatekeeping and exclusion.
- **Takeaway:** ZKPs are not a silver bullet for age verification: they must be embedded within a broader system context (usability, revocation, interoperability, user autonomy).

However...

- Standardisation bodies and civil society are thinking about it
 - Workshops at the IETF and W3C
- The goal is to preserve privacy but not enhance exclusion or censorship in the process
- See: <https://brave.com/blog/zkp-age-verification-limits/> ,
<https://www.ietf.org/ietf-ftp/slides/slides-agews-paper-private-and-decentralized-age-verification-architecture-00.pdf>



THANK YOU!

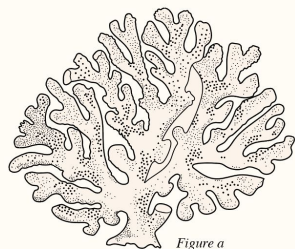


Figure a
Acropora

Coral

@claucece

<https://sofiaceli.com/>